Enhancing predictive maintenance architecture process by using ontology-enabled Case-Based Reasoning

Juan José Montero-Jiménez
Tecnológico de Costa Rica
Cartago, Costa Rica
ISAE-SUPAERO
Toulouse, France
Email: juan.montero@itcr.ac.cr

Rob Vingerhoeds
ISAE-SUPAERO, France
Toulouse, France
Email: rob.vingerhoeds@isae-supaero.fr

Bernard Grabot
ENIT
Tarbes, France
Email: bernard.grabot@enit.fr

2021 IEEE International Symposium on Systems Engineering (ISSE) | 978-1-6654-3168-2/21/$31.00 ©2021 IEEE | DOI: 10.1109/ISSE51541.2021.9582535

***Abstract*—A common milestone in systems architecture development is the logical architecture. It provides a detailed overview of the system components and their interfaces but keeps the architecture as generic as possible, meaning that no component is bound to a specific technology. Subsequently, the architect searches for physical/informational components to fulfill the logical architecture and can apply structured creativity to look for innovative solutions. This search can turn out to be a difficult and long-lasting task depending on the system complexity. Too many options may be available to fulfill the logical system components and not always the most suitable ones are identified. This problem is for instance encountered in the design of new predictive maintenance systems, especially when selecting the components to carry out the diagnostics and prognostics. The current study proposes to support the choice of suitable components combining case-based reasoning and ontologies. A domain ontology has been developed as a terminology framework to support the case base, case structure and similarity measures for a case-based reasoning Decision Support System (DSS). The DSS uses attributes of the new problem to solve and suggests the most similar cases from past experiences. The retrieved solutions can be adapted to develop a new predictive maintenance architecture. The decision support system has been tested with data coming from proved predictive maintenance solutions documented in scientific publications.**

***Index Terms*—System architecture, case-based reasoning, predictive maintenance, knowledge reuse, structured creativity, decision-support system.**

I. Introduction

The life cycle of complex systems is composed of several stages starting from the concept phase [1], which can be seen as a very crucial stage. Following a systems engineering approach, the concept phase starts by gathering the needs and desires from all stakeholders, that are then translated into a formal set of requirements to be classified and prioritized to facilitate their analysis. Once the set of stakeholders' requirements is agreed by all parties a creative process starts to find the solution or solutions that meet the requirements. This is then formalized in a system architecture before a detailed design and the system implementation start in the development stage. The system architecture formally describes and represents the system elements and their relationships [1].

Developing a system architecture can be a complex task in which many options are available. Several methods exist to carry out this architecture process. Very often these methods cover a functional analysis of the system that includes a functional decomposition allowing to handle complexity [1]–[3]. Logical components are proposed to fulfill each sub-function. Once the logical components are defined, physical/informational components are selected and allocated to the logical components to complete the systems architecture.

Proposing suitable components to fulfill the logical architecture is an important challenge for the system architect. Sometimes, the architect may have an immediate vision of a solution, yet this does not always lead to the most efficient one. As the development of complex systems may represent an important investment, structured creativity is increasingly used in the concept stage [2]. This means the architect explores the solution space of the system using a structured methodology, identifying and proposing several possible solutions and performing trade-off analysis to identify the most suitable ones. Exploring the solution space of a new system can be long-lasting task, specially when the architect has many options for suitable components. Also, no explicit design rules exist to perform such component selection.

To overcome this challenge, this study proposes a searching tool that combines Case-Based Reasoning (CBR) and Ontologies. CBR is a problem solving paradigm based on previous experiences that can be used for design purposes. Ontologies are semantic knowledge representations that can model the terminology of a specific domain. The proposed approach aims at allowing the architect to save time when exploring the solution space and at the same time providing inspiration by offering diverse possible solutions to fulfill the logical components. In contrast to other implementations of CBR for systems design [4], [5], the proposed tool incorporates a domain ontology that serves as terminology framework for the CBR system. It helps to model the case structure, store the case base and compute semantic similarity for retrieval purposes in CBR.

This paper is organized as follows: section II explains the context of the research by introducing the building blocks of the proposed framework: Case-Based Reasoning (CBR), ontologies, and ontology-enabled CBR systems. Section III presents the design of predictive maintenance systems and

their architectures, domain to which the proposed approach is applied. Section IV explains the proposed approach of the use of an ontology-enabled CBR system to enhance structured creativity. Section V presents the implementation of this approach for predictive maintenance systems design. Section VI describes the results of the resulting Decision Support System (DSS) and section VII concludes the paper by summarizing the lessons learnt and providing perspectives of future work.

II. Background

The current study aims at implementing an ontology-based CBR system that can be used to retrieve suitable components for predictive maintenance (PdM) systems. The objective is to propose to system architects a means to explore the solution space more efficiently so not to miss important potential solutions. This section provides a background of CBR, ontologies, and their combination in recent knowledge modelling and reuse works.

A. Case-based Reasoning (CBR)

Case-based reasoning is a paradigm that leverages past problem solving experience, in form of concrete solving cases, when it comes to solving new problems [6]. Case-based reasoning tries to implement this way of reasoning. The following definition for CBR is used here:

Solve a new problem by remembering a previous similar situation and by reusing information and knowledge of that situation.

Case-based reasoning is a paradigm in which specific knowledge of previously experienced problem situations is being used to solve a new problem, by finding close previous cases, adapting and reusing those previous experiences. It leads to a form of incremental, sustained learning, where information from new situations is kept for future use.

Solving a problem with CBR is performed in a cyclic process of several steps, the so-called CBR-cycle [7], triggered by a new problem: solving the target case. The first phase aims at retrieving the most similar cases from a knowledge base that stores all previous cases. The target case is compared to the stored cases in a the case base using different similarity measurements. The most similar retrieved case is proposed as possible solution in the reuse phase. Some adaptation may be needed to implement the solution for the target case. Subsequently, revision phase takes place to ensure that the suggested solution achieves to solve the problem. Once the solution is validated, in a last phase it is stored in the knowledge base so that it can be reused in future similar problems.

B. Ontology

In information science, an ontology is a formal explicit description of concepts in a domain of discourse, properties of each concept describing its features, attributes and restrictions [8]. One of the goals in developing ontologies is "sharing a common understanding of the structure information among people and software agents" [9]. This means that the vocabulary used by people in a specific domain of knowledge is "machine readable". All concepts in an ontology are represented by classes which are linked by properties (also called relations). Ontologies are built using formal languages. One of the most recognized languages is the Web Ontology Language in its second version (OWL2) which is recommended by the World Wide Web Consortium (W3C) [10] because of its importance for the semantic web. The semantic web is an extension of the current web, where the information is well-structured and well-defined so that it can be processed by a machine [11].

Ontologies in OWL2 are compatible with information written in Resource Description Framework (RDF) [10]. In RDF, information is represented in semantic triples: subject-predicate-object. OWL2 ontologies are primarily exchanged as RDF documents; all the knowledge stored in an ontology can be also represented by semantic triples. Two related classes will represent the subject and the object of the triple, while the relation between the two classes represents the predicate of the triple. RDF structure provides a flexible means to model, storage and manage information [10]; other methods requiring variable-length fields would require a more complicated implementation.

Ontologies can help to make the knowledge explicit, defining the relations among different terms. This allows deep analysis on domain knowledge, helping to identify semantic rules among the terms. These rules can be used to develop algorithms that perform automated inferences based on the domain knowledge.

C. State-of-the-art of ontology-enabled CBR

For any CBR application, a very important step is the definition of a domain vocabulary that will serve to model the cases, the case base, the similarity measures and the solution adaptation knowledge [12], [13]. CBR was formalized before ontologies gained importance in the artificial intelligence field. Recent research trends incorporate ontologies to facilitate natural language modelling and processing. For CBR, ontologies can be used to enhance case indexing and retrieval, to improve semantic similarity estimation, to improve case representation, to improve case adaptation, and to improve case retention in a case base [14]. Ontologies provide the terminology framework to better describe the case attributes. In [15], an ontology was used as vocabulary framework for a case-based reasoning system that automates emergency response services. The authors use the ontology in the information extraction process, during the lexical analysis. Later, the ontology is used in the case retrieval by helping to determine the similarities for the case attributes.

Ontologies provide the possibility to compute the similarity between two semantic terms by the use of the ontology-based similarity. As [16] explains, ontology-based similarity can be determined by two different manners: by computing the ontological distance or by comparing the features of the terms. As ontologies can be represented by graphs in which the

terms are in the nodes and the relations among the terms are the edges, the ontological distance between two terms is the shortest edges path from one term to another. Feature-based similarity is obtained by comparing the properties of the classes. This feature based similarity can vary depending on the scope. Selecting the different features will yield to different similarities.

Ontologies can also be used to model the cases and the case base in CBR. The ontology classes can be instantiated with the different cases of the case base. An instantiated ontology can be be referred to as a knowledge base of the a specific domain [8]. This knowledge base stores the information in the Resource Description Framework (RDF) which can be easily retrieved and managed. In [17], an ontology is presented to support a CBR system for mechanics tolerance specification in which the case base is built on top of the ontology. Another example uses an ontology to build the case base of a CBR system for decision support on manufacturing process selection [18]. Specifically in systems design, [4] proposes an integrated approach using an ontology and a CBR system to propose design solutions based on the initial requirements and preferences. The authors propose a generic design approach in which the cases are built from requirement attributes and the solution obtained from previous experiences with similar requirements.

Current trends in systems architecture aim at incorporating structured creativity methods to explore the solution space in the concept phase of complex systems. The current study aims at integrating CBR and a domain ontology to facilitate the architect work of retrieving suitable components for predictive maintenance systems.

III. Predictive maintenance systems

Within the maintenance strategies to trigger maintenance actions, three terms are commonly used: corrective maintenance, preventive maintenance and predictive maintenance. Corrective maintenance triggers the maintenance actions once the failure of a component or system has occurred. Preventive maintenance trigger the maintenance actions by using fixed operation intervals of the component or system; such as time, cycles, kilometres, flights, among others. Predictive maintenance (PdM) aims at determining the right moment to trigger the maintenance actions based on the condition of the maintainable system under consideration [19]. Current fast expanding tools and technologies such as machine learning, internet of things, Industry 4.0, big data, boost predictive maintenance implementation to reach safer, more reliable and more efficient technical systems. Predictive maintenance is often studied within the disciplines of Condition-Based Maintenance (CBM) and Prognostics and Health Management (PHM).

Predictive maintenance is carried out by specialized systems whose main function is to estimate the precise moment to trigger maintenance actions. This main function can be decomposed into six different sub-functions (Figure 1: collect data, pre-process data, detect faults, assess degradation, compute remaining useful life and make recommendation report). This functional decomposition is widely used to develop the architecture of predictive maintenance systems.A new predictive maintenance system may require components to fulfill all sub-functions or a smaller subset of them [20]. For example, a system can be intended only to cover a fault detection, then the architecture should have components to collect data (F1), pre-process data (F2), detect the fault (F3) and make a recommendation report (F6).

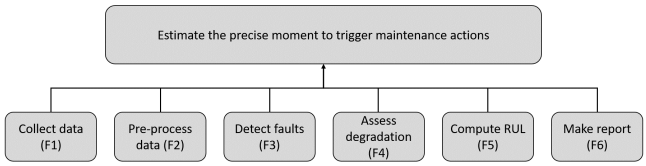

Fig. 1. Functional decomposition for a predictive maintenance system. [20]

Specifically for the diagnosis and prognosis sub-functions, there exist several models that can fulfill the requirements. These models can be implemented individually following a single model approach for each sub-function or many of them can be combined in a multi-model approach [19]. These models can be divided into three main families:

- *Knowledge-based models*: these models are built from expert knowledge and experience from which it is possible to explicitly define rules, cases and constraints that can be used to perform reasoning. These explicit models highly rely on the access to experts which can be a challenge for their development.
- *Physics-based models*: these models use the laws of physics to assess the degradation of components. They demand high skills on mathematics and physics of the phenomena for the application.
- *Data-driven models*: these models use data records that often consist of time series. Data-driven models have gained a lot of importance in recent years thanks to the improved availability of computational power and the large amounts of data produced every day by technical systems.

Predictive maintenance systems accurately illustrate the problem for system architects to select suitable components that fulfill the logical architecture. There are no explicit rules that guide the architect throughout the components selection to fulfill the logical architecture when developing a new predictive system. There exist several models that can be used to carry out the diagnosis and prognosis functions of the system; and these models are often combined as one single model hardly addresses a single task of a predictive maintenance system [19]. A decision support system can help the architect to select the model or combination models that have been used in the past accentuates the need for supporting architects for components selection.

IV. Ontology enabled Case-Based Reasoning for systems architecture

When developing a new system architecture, the architect often looks for inspiration from previous experiences and related systems that already exist. The idea to integrate Case-Based Reasoning (CBR) and ontologies for systems architecture comes from an analogy between CBR and the structured creativity process, as well as the need to formally model the domain vocabulary that allows the knowledge reuse. This section explains the analogy between CBR and structured creativity. A first concept of the integration of a ontology-enabled CBR Decision Support System (DSS) is introduced as means to help the architect to look for inspiration from previous experiences. The analogy and the concept of the DSS, here intended for predictive maintenance systems, are generic and can be used for the development of other types of complex systems as well.

A. The analogy between structured creativity and CBR for systems architecture

A common milestone in the systems architecture development is the logical architecture. Structured creativity at the logical architecture is based on the combination of different possible physical/informational components that can fulfill the logical components. This allows the exploration of the solution space and may help the architect to identify innovative solutions for a new system. If several possible solutions are identified a trade-off analysis may be necessary to select the most suitable one. The selected architecture serves as a basis for detailed design of the systems. The knowledge gathered from the new implemented system can be used by the architect to develop future systems.

There is an analogy between the structured creativity work performed by the architect to select suitable components to fulfill the logical architecture and the four phases of Case-Based Reasoning: retrieve, reuse, revise and retain. A graphical representation of this analogy is presented in Figure 2 (in Capella notation [3]). All activities before the logical architecture are outside the scope for the current study and for layout simplification, these activities have been summarized in a single box on Figure 2.

- The proposed analogy relates the retrieve phase of CBR with the search performed by the architect on previous related systems that may serve as inspiration for the development of the new system.
- The reuse phase of CBR would be the allocation of the identified components to fulfill the new system architecture.
- The revise phase of CBR would start by the trade-off analysis done by the architect to identify the most suitable components. This phase continues until the verification and validation of the implemented system which are usually performed by other actors than the architect.
- After validation of the new system, the architect may keep the records to be used in the future; this would be the retain phase of CBR.

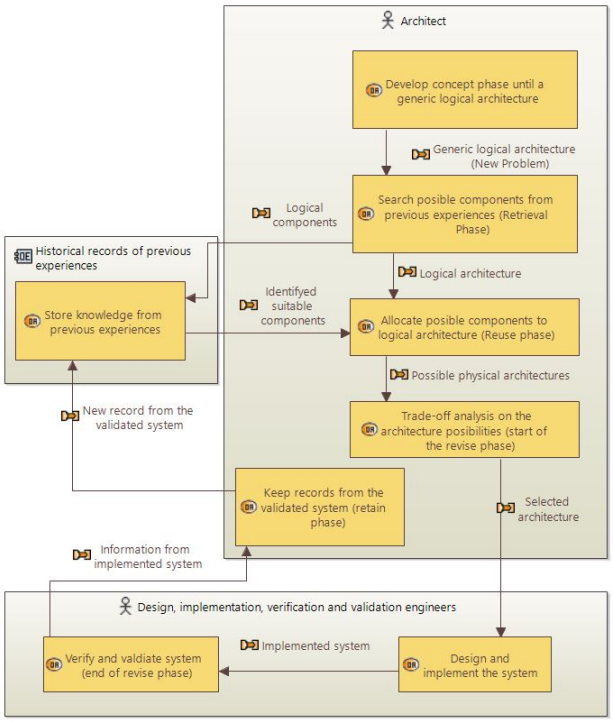

Fig. 2. Analogy between case-based reasoning and the tasks performed by a systems architect, in Capella notation [3]

This analogy remains generic in terms of the system to be developed. All the activities are carried out "manually" by the architect and the records from previous experiences may not be structured to facilitate their reuse. This manual approach may work when the amount of previous experiences is limited so that the solution space to be explored by the architect remains manageable. When the number of previous experiences is high or the requirements complex, the selection of the most suitable components may not be easily achieved. Retrieving knowledge from an important number of previous architectures may consume too much time and important options may be missed.

B. A Decision Support System concept using CBR and ontologies

Considering this analogy between CBR and the structured creativity work, different algorithms can be proposed to support the architect in the different phases of the architecture work. In a first step, the current research focused on the development of a Decision Support System (DSS) able to perform the search and recommendation of suitable physical/informational components. This can help the architect save time in the concept phase, and allows a broader analysis done by machines, analysis that can be hardly addressed by humans.

Figure 3 presents a concept of the DSS and how it fits in the analogy presented on Figure 2. This concept is composed of three main parts: the case base, the retrieve engine, and the domain specific ontology. The case base stores the cases from the past in a structured format to facilitate reuse. The

architect will present to the retrieve engine the information from the new system under development and will obtain a set of the most similar cases retrieved from the case base that will serve as inspiration when selecting suitable components for the logical architecture.

Within this structure the ontology plays a vital role. The cases, attributes of these cases and the similarities among the different text based variables are often described in natural language. Modeling this natural language and making it machine readable is necessary to automate the case retrieval.

V. Implementation on the Predictive Maintenance Case Study

The proposed approach assumes that the logical architecture of the system has been developed. A systematic approach to develop new predictive maintenance systems has been proposed in [20]. The ontology-enabled CBR Decision Support System (DSS) proposed here is for predictive maintenance models (components) selection to fulfill the logical architecture of a new system, especially to fulfill the diagnosis and prognosis functions. The stakeholder requirements for the DSS are as follows:

1) The system shall suggest suitable models to fulfill a specific diagnosis and prognosis function.
2) The system shall provide a list of diverse potential solutions for each logical component.
3) The system shall provide a structured framework to store the knowledge form previous predictive maintenance systems.
4) The system shall base the recommendation on known attributes of the new predictive maintenance system such as function to fulfill, the maintainable system and the available condition data from the maintainable system.
5) The system shall provide design information from the suitable models such as: performance indicators, input variables and models configuration.

A. Domain ontology development

The ontology for this study was developed using the methodology *Ontology Development 101* [8]. To delimit the scope of the ontology a set of competency questions should be considered. The ontology must be able to answer these questions. In this study, these competency questions have been defined in cooperation with experts in predictive maintenance and aim at gathering useful information that can be used in the design of new predictive maintenance systems. In a first attempt, the scope of the ontology is limited by the following competency questions:

- What are functions of a predictive maintenance system?
- What are the systems on which predictive maintenance has been implemented?
- What models have been used to fulfill each function of the system?
- For a given predictive maintenance case, is the predictive maintenance system implemented online or off-line?
- What performance indicators have been used to assess a model that fulfills a predictive maintenance function?
- What data is analyzed by the predictive maintenance models?
- If several models are being used to fulfill a specific function, what is the models configuration in the system?
- Where is the predictive maintenance implementation documented?

The ontology model was developed using a top-level domain-neutral ontology, the Basic Formal Ontology (BFO) [21]. It also uses a set of mid-level domain-neutral ontologies, the Common Core Ontologies (CCO) [22]. BFO and CCO

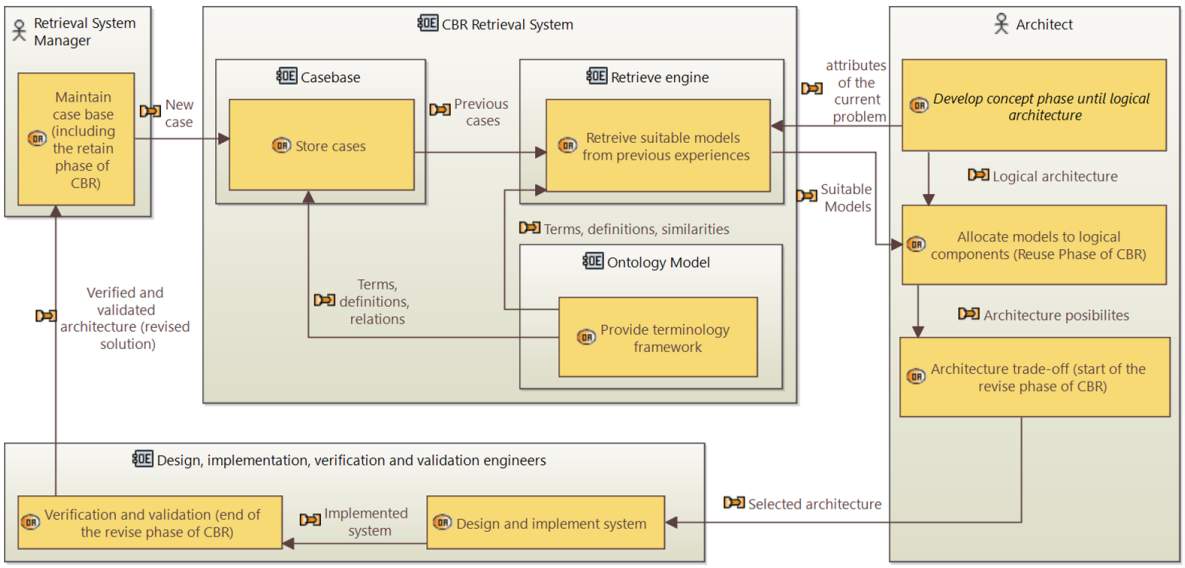

Fig. 3. Incorporation of an CBR retrieve system to facilitate logical components selection in the architecture process

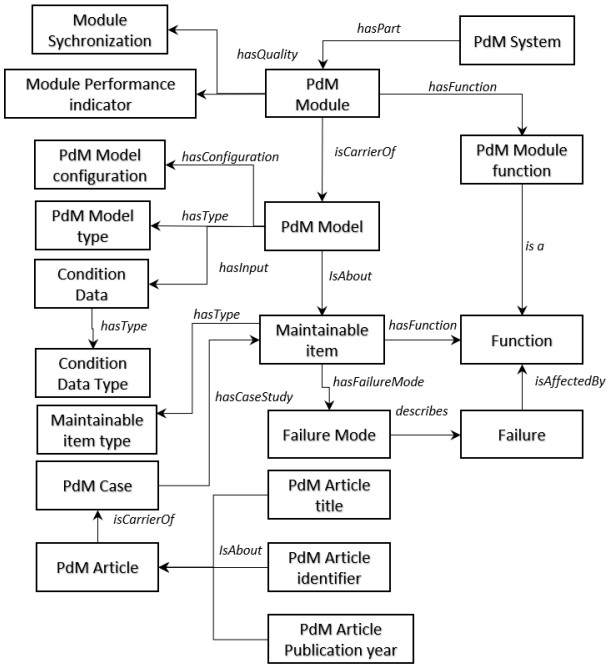

Fig. 4. Classes and relations in the ontology to support CBR for PdM component selection

have also been used in other ontologies in the industrial domain. The standardization on the developed ontology by adopting these upper level ontologies facilitates its future reuse and integration with other ontologies in the industrial domain. The Ontology was created using `Protégé©` (version 5.5.0) and its consistency was verified using the ontology reasoner HermiT OWL (version 1.4.3.456).

Figure 4 summarizes the most important classes and relations in the ontology for CBR decision support system for predictive maintenance components selection. For the validation of the current approach, the ontology was populated with information coming from the publications considered in a recent structured literature survey about predictive maintenance [19]. The OWL API [23] was used to populate the ontology and to provide the classes and methods for the integration of the ontology and the CBR system.

B. CBR decision support using MyCBR SDK

MyCBR is an open-source similarity-based retrieval tool for Case-Based Reasoning (CBR) [12] that offers two possible options for CBR solutions: a stand-alone application called myCBR Workbench and a Software Development Kit (SDK). The SDK is written in Java and includes all classes and methods to develop CBR applications and integrate them to other systems.

When developing a CBR retrieval system using myCBR, the first step is to determine the case structure. Cases are represented by a coupled vector of attributes Case = [Problem attributes, Solution Attributes]. The problem attributes are used to measure the similarity of a target case and the cases stored in the case base. The solution attributes are stored as part of the solution information for the cases. Taking as reference the classes in the domain ontology, for predictive maintenance design the case vector will have the following structure:

- **`Problem Attributes`** = [`PdM Function`, `Maintainable Item`, `Maintainable Item Type`, `Condition Data Type`, `Module synchronization`, `PdM Article Publication year`]
- **`Solution Attributes`** = [`PdM Model`, `PdM Model Configuration`, `PdM Model Type`, `Module Performance Indicator`, `PdM Article Identifier`, `PdM Article Title`]

Once the case structure is defined, a similarity measure must be assigned to each problem attribute. In myCBR the similarity for each problem attribute is referred to as local similarity. MyCBR offers a complete set of similarity measures to compute local similarities. For the current study three main types of similarities have been used: integer/float similarity, symbol similarity and string similarity [12]. The domain ontology plays a vital role in the computation of some of the local similarities. For example, for the predictive maintenance (PdM) functions an ontological similarity approach based on the ontological features is adopted to obtain the similarity among the options [16]. This similarity is computed based on the models that have been historically used to fulfill each function. All the cases stored in the ontology are used to compute this similarity and if the case base is updated with new cases in the future, the similarity will be automatically updated. This similarity measure was selected for the attributes `PdM Function`, `Maintainable Item Type`, `Condition Data Type`, and `Module synchronization`.

After the similarity computation for each attribute of the problem, the combination of all these individual similarities in a global similarity takes place. Each attribute has a weight and an amalgamation function calculates the final similarity based on local similarities and weights. MyCBR offers two different options to compute the global similarity: weighted sum and Euclidean distance.

- `Weighted sum`: the sum of similarities considering the weight of each one (equation 1).

$$sim_{global} = \sum_{i=1}^{n} w_i \cdot sim_i \tag{1}$$

  The values of n, w_i and sim_i are respectively the number of variables, the weight and the similarity of variable i.
- `Euclidean distance`: distance between two points in Euclidean space. It represents the length of a line segment between the two points (equation 2).

$$sim_{global} = \sqrt{\sum_{i=1}^{n} w_i \cdot sim_i^2} \tag{2}$$

For both amalgamation functions there is a re-scaling normalization between 0 and 1, with 1 being the maximum global similarity between the target case and a retrieved case. This

maximum global similarity is achieved when the target case attributes are identical to those of the retrieved case. This maximum global similarity can be achieved independently from the number of problem attributes that are introduced to the DSS. If the architect uses only on two or three attributes to describe the target case, a global similarity of 1 can be obtained based on those two or three attributes.

VI. Results and discussion

The case base stored in the domain ontology was instantiated with 263 cases obtained from the 135 research papers consulted in [19]. For a cross-validation of the ontology-enabled CBR Decision Support System (DSS), the cases were randomly divided into two sets. A set of 200 cases was used as case base in the ontology, the other 63 cases were used as test set.

A. Verification

The DSS verification checked how each local similarity performs. For this, several searches were realized using a reduced set of problem attributes. The verification results are described hereafter:

- With searches using one single attribute it was possible to validate the similarity measures assigned to the problem attributes: `PdM Function`, `Maintainable Item`, `Maintainable Item Type`, `Condition Data Type` and `Module synchronization`. These are the variables that the user can directly introduce as inputs for the DSS. When presenting only one out of these five possible attributes to define the target case, the DSS retrieved with similarity of 1 every case from the case base that shared the same attribute value as for the target case, independently from the assessed attribute.
- When increasing one by one the number of attributes of the target case, this behaviour was consistent. Several combinations using a reduced set of attributes were tested to confirm the DSS consistency to retrieve the most similar cases.
- The more attributes shown to the DSS, the fewer cases with similarity of 1 or close to one were retrieved. This is an expected behaviour because the stored cases in the case base are diverse. It was rare to find two cases in the case base with the same problem attributes and when it happened, the solution attributes of the retrieved cases were completely different.
- For some of target cases in the testing set, the maximum similarity reached was between 0.7 and 0.8 proving that when no identical cases were found an thus, the maximum global similarity of one can not be reached.
- The searches using a reduced number of attributes also helped verify the similarity measure implemented for the `PdM Article Publication year`, the only attribute that the user does not specify in the DSS to describe the target case and it is used to compute the global similarity. The current year is automatically retrieved from the operative system and it is compared against the year of publication of each PdM article. Models applications in newer papers are favored over similar applications from the past. With a reduced number of problem attributes it was possible to confirm that cases from recent publications had a higher similarity to the target case compared to those from older publications.

B. Validation

The cross-validation of the DSS checked that the stakeholder requirements of section V were met.

- Requirement 1 states that the DSS must suggest suitable components to fulfill diagnosis and prognosis functions in predictive maintenance systems. In practice, an architect needs at least one suitable model to develop the architecture but may look for inspiration from the retrieved cases with highest similarity. In the first attempt the acceptance criteria for the DSS is to provide at least 1 suitable model that can fulfill the PdM function of the target case. For each of the 63 cases it was possible to retrieve the similarity to the 200 cases in the case base. A list of the 5 most similar cases for each target case in the testing set was retrieved. For all the test cases at least one suitable model with similarity above 0.7 was retrieved. For each test case, the implemented PdM model is known. In 40% of the tested cases the implemented PdM model matched to one of proposed solutions by the DSS. In 40% of the tested cases all the proposed solutions by the DSS were implemented for the same PdM function. For the other 60% there was at least one recommended PdM model that was originally used for a different PdM function but being possible to adapt it to the target case. For example, for a target case to fulfill a fault detection function, the DSS proposed a classification model that was used for fault identification in the retrieved case. Classification techniques are suitable for both PdM functions. With these results, the acceptance criteria of the first requirement has been fulfilled; the capability of ontology-enabled CBR decision support system to recommend suitable models to fulfill a specific function of a predictive maintenance system has been confirmed.
- The validation highlights an important point of improvement with regard to the diversity in the retrieved cases. This is related to the second requirement. In at least half of the tested cases, within the list of the 5 most similar cases retrieved by the DSS, two or three cases proposed the same PdM model as potential solution. It is important to notice that this is not a problem of repeated cases in the case base, the similarities of the retrieved cases are different among them but sometimes they propose the exact same solution. Diversity in retrieved cases is a common problem in CBR [13], especially when several different potential solutions are expected from the retrieval phase. Diversity in the retrieved cases can help architects to develop innovative solutions. The identified problem narrows down the solution space recommended

by the ontology-enabled CBR system to the architect and represents an improvement of the DSS for future work.
- As the case base is stored in the domain ontology the third requirements is met. The ontology is a formal and structured knowledge representation that stores the predictive maintenance implementations of the past that can be reused.
- The case structured in the DSS was determined in such a way to fulfill requirements 4 and 5. The problem attributes are based on known parameters of the new predictive maintenance system and the solution attributes of the retrieved cases provide useful information for the design of the system.

For the verification and validation of the proposed DSS, both amalgamation functions were tested with no significant differences in the results. For this current study, all the attributes were assumed to have the same importance, so no special weights were assigned to the local similarities. As such they had a default value of 1.

VII. Conclusion and future work perspectives

The current study presents the use of a ontology-enabled Case-Based Reasoning (CBR) recommendation system to select suitable components for predictive maintenance systems architecture. The proposed approach benefits from the semantic knowledge modelled using a domain ontology. It allows to build the similarity measures for different attributes of the CBR system. The ontology stores the case base for the CBR that was created using the myCBR platform. The validation shown that the ontology-enabled CBR system is capable to retrieve suitable components to fulfill specific predictive maintenance functions. The retrieval capability was validated with all problem attributes that have been introduced individually and combined to the recommendation system. The retrieved cases can help architects as inspiration to develop innovative solutions for new predictive maintenance systems.

Future work perspectives include the refinement of the case retrieval functions, case base maintenance to improve the diversity in the retrieved cases, and the integration of a trade-off analysis that can help the architect perform the selection among the retrieved components. Future perspective also include the incorporation of algorithms to address other CBR phases such as adaptation of the retrieved cases for the target problem and case base maintenance after retaining newer cases. As the analogy of CBR and structured creativity was proposed as a generic approach, its implementation in other case studies is also considered in the future work perspectives.

Acknowledgment

The authors would like to thank the students Johanna Mazouzi from ISAE-ENSMA, Augusto Miyagawa and Hugo Muñoz-Hernández from ISAE-SUPAERO, for their collaboration in this research.

References

[1] INCOSE, *Systems Engineering Handbook. A guide for system life cycle processes and activities. Fourth Edition.* Wiley, 2015.
[2] E. Crawley, B. Cameron, and D. Selva, *System Architecture: Strategy and Product Development for Complex Systems*. Pearson Higher Education, Inc., 2015.
[3] P. Roques, *Systems Architecture Modeling with the Arcadia Method 1st Edition*. ISTE Press, 2018.
[4] J. C. Romero Bejarano, T. Coudert, E. Vareilles, L. Geneste, M. Aldanondo, and J. Abeille, "Case-based reasoning and system design: An integrated approach based on ontology and preference modeling," *Artificial Intelligence for Engineering Design, Analysis and Manufacturing: AIEDAM*, vol. 28, pp. 49–69, 2014.
[5] A. Tidemann, F. O. Bjørnson, and A. Aamodt, "Case-based reasoning in a system architecture for intelligent fish farming," in *Frontiers in Artificial Intelligence and Applications*, 2011.
[6] J. Kolodner, *Case based reasoning*. Morgan Kaufmann, 1993.
[7] A. Agnar and E. Plaza, "Case-Based reasoning: Foundational issues, methodological variations, and system approaches," *AI Communications*, vol. 7, no. 1, pp. 39–59, 1994.
[8] N. F. Noy and D. L. McGuinness, "Ontology Development 101: A Guide to Creating Your First Ontology," Tech. Rep., 2001.
[9] T. R. Gruber, "A translation approach to portable ontology specifications," *Knowledge Acquisition*, vol. 5, no. 2, pp. 199–220, 1993.
[10] World Wide Web Consortium, "OWL 2 Web Ontology Language," 2012. [Online]. Available: http://www.w3.org/TR/2012/REC-owl2-overview-20121211/
[11] D. L. Nuñez and M. Borsato, "OntoProg: An ontology-based model for implementing Prognostics Health Management in mechanical machines," *Advanced Engineering Informatics*, vol. 38, pp. 746–759, 2018.
[12] K. Althoff, T. Roth-Berhofer, K. Bach, and C. Severin, "Documentation: myCBR," 2006. [Online]. Available: http://mycbr-project.org/%0Adownloads/myCBR_3_tutorial_slides.pdf
[13] R. L. De Mantaras, D. Mcsherry, D. Bridge, D. Leake, B. Smyth, S. Craw, B. Faltings, M. L. Maher, M. T. Cox, K. Forbus, M. Keane, A. Aamodt, and I. Watsoni, "Retrieval, reuse, revision and retention in case-based reasoning," *The Knowledge Engineering Review*, vol. 20, no. 3, pp. 215–240, 2005.
[14] J. Prentzas and I. Hatzilygeroudis, "Combinations of case-based reasoning with other intelligent methods," in *CEUR Workshop Proceedings*, 2008.
[15] K. Amailef and J. Lu, "Ontology-supported case-based reasoning approach for intelligent m-Government emergency response services," *Decision Support Systems*, vol. 55, pp. 79–97, 2013.
[16] D. Sánchez, M. Batet, D. Isern, and A. Valls, "Ontology-based semantic similarity: A new feature-based approach," *Expert Systems with Applications*, vol. 39, pp. 7718–7728, 2012.
[17] Y. Qin, W. Lu, Q. Qi, X. Liu, M. Huang, P. J. Scott, and X. Jiang, "Towards an ontology-supported case-based reasoning approach for computer-aided tolerance specification," *Knowledge-Based Systems*, vol. 141, pp. 129–147, 2018.
[18] M. M. Mabkhot, A. M. Al-Samhan, and L. Hidri, "An ontology-enabled case-based reasoning decision support system for manufacturing process selection," *Advances in Materials Science and Engineering*, 2019.
[19] J. J. Montero Jimenez, S. Schwartz, R. Vingerhoeds, B. Grabot, and M. Salaün, "Towards multi-model approaches to predictive maintenance: A systematic literature survey on diagnostics and prognostics," *Journal of Manufacturing Systems*, vol. 56, pp. 539–557, 2020.
[20] J. J. Montero Jiménez and R. Vingerhoeds, "A System Engineering Approach to Predictive Maintenance Systems: from needs and desires to logical architecture." in *5th IEEE Int. Symposium on Systems Engineering 2019,*, Edinburgh, 2019.
[21] International Organization for Standardization (ISO), *ISO/IEC 21838-2 - Information technology - Top-level ontologies (TLO) - Part 2: Basic Formal Ontology (BFO)*, 2020.
[22] R. Rudnicki, "An Overview of the Common Core Ontologies 1.3," Buffalo, NY, p. 29, 2020. [Online]. Available: https://www.nist.gov/system/files/documents/2019/05/30/nist-ai-rfi-cubrc_inc_004.pdf
[23] I. Palmisano, "OWLAPI Documentation," 2020. [Online]. Available: https://mvnrepository.com/artifact/net.sourceforge.owlapi/owlapi-distribution/5.1.17